BOTSWANA INTERNATIONAL UNIVERSITY OF SCIENCE AND TECHNOLOGY

Department of Computing and Informatics

GUESTHOUSE DISCOVERY PLATFORM

An Airbnb-Inspired Guesthouse Discovery Application

Group Project Documentation
Admin Web Application • Customer Mobile Application

Group Project — Guesthouse Discovery Platform

Instructor: Ontiretse Bagwasi

Group Members

Group Member	Student Number
Tsholofelo Blessing Bathusi	24019716
Peo Abigail Motswasele	24020099
Pako Tumelo Ikobeng	24019736
Helen Tlhomamo Mahule	24019725
Angel Mhaladi	24019850
Lefa Kato Ramonameng	24020066
Pako Serumola	24019735

1. Introduction

Our project is a Guesthouse Discovery Platform: an app that lets people find and compare guesthouses in a given area, paired with a web portal that admins use to manage those listings. It's inspired by how Airbnb presents accommodation, but we are not building a booking system. There is no payment, checkout, or reservation anywhere in this project customers can only browse, search, favourite, rate and comment.

The idea came from a simple problem: guesthouse information is usually scattered across Facebook pages, word of mouth, and outdated websites, so it's hard to quickly compare price, location and amenities. Our platform pulls that information into one place. Admins capture guesthouse details once through the web portal, and customers see the same data instantly on the mobile app, because both applications talk to the same backend and database.

2. Aim and Objectives

Aim: to build a two-part guesthouse discovery system, a mobile app for customers and a web portal for admins that makes it easy to find, compare and interact with guesthouse listings without any booking functionality.

Objectives

1. Let customers browse and search guesthouses without needing an account.
2. Support sorting by price and rating, with the cheapest guesthouse shown first by default.
3. Allow logged-in customers to favourite, rate and comment on listings.
4. Give admins a portal to register guesthouses and create other admin accounts.
5. Build one shared backend and database so both apps stay in sync.
6. Add a dashboard summarising key stats for admins.

3. Scope and Limitations

In Scope

Mobile app: browsing without login, search by name/location, sort by price and rating (cheapest first by default), full guesthouse details with gallery and map, customer login, favourites, ratings and comments.

Admin portal: admin login, creating extra admin accounts, registering guesthouses (name, logo, gallery, description, amenities, coordinates, price, city, country, contact), editing listings, and a dashboard with totals, top city, and top-rated guesthouse.

Out of Scope

No booking, no room reservations, no online payments, and no room availability management. The system only presents listings for discovery it doesn't handle the transaction side of things at all.

Limitations

- Guesthouse owners can't register listings themselves yet only admins can, for this version.
- Personalised recommendations and similar "nice-to-have" features are left for future work, not this release.

Proposed Enhancement: Google Sign-In

The brief only requires standard email/password login for customers. As an optional addition on top of that, we're proposing Google sign-in as a second login option, since it's a common convenience feature and it's straightforward to add with Firebase Authentication's free tier. This is not a required feature it's something we'd like to add if time allows, and the standard email/password login covers the brief either way.

4. Stakeholders and Target Users

Stakeholder	Role
Customers	Search, browse, favourite, rate and comment on guesthouses
Admins	Register guesthouses, manage admin accounts, check dashboard stats
Guesthouse owners	Provide their establishment's info to an admin for listing
Our team	Design, build, test and document the system

5. Functional Requirements

These cover what the assignment brief asks for. Each one should be checkable against the finished system.

Requirement
An administrator can log in using valid admin credentials.
A logged-in administrator can create additional admin accounts.
A logged-in administrator can register a guesthouse, including name, logo, gallery images, description, amenities, map coordinates, price per night, city, country and contact details.
A logged-in administrator can edit an existing guesthouse listing.
The dashboard shows the total number of registered guesthouses.
The dashboard shows which city has the most registered guesthouses.
The dashboard shows the most highly rated guesthouse.
A customer can browse guesthouses without creating an account.
A customer can search for a guesthouse by name or by location.
Search results can be sorted by price or by rating.
The cheapest guesthouse is shown first by default.
A customer can register an account and log in.
A logged-in customer can favourite a guesthouse.
A logged-in customer can rate and comment on a guesthouse.
Guesthouse location is shown on a map on both applications.
The mobile app always reflects the same data managed through the admin portal.

Proposed enhancement (not required by the brief): a customer can also sign in using a Google account, as an alternative to email/password login.

6. Non-Functional Requirements

Category	Requirement
Usability	A first-time user should be able to search for and view a guesthouse without needing instructions.
Performance	Search results should return within a few seconds under normal conditions.

Category	Requirement
Security	Passwords are hashed, and admin-only actions require a valid login token.
Reliability	The mobile app should always reflect the current data in the database.
Scalability	The data model should handle more guesthouses, users and ratings without major changes.
Maintainability	Code is split into clear modules (auth, guesthouses, ratings, favourites) so the team can work in parallel.

7. User Stories

Actor	Story
Customer	I want to browse guesthouses without an account so I can check what's available before signing up.
Customer	I want to search by location so I can find guesthouses in the area I'm visiting.
Customer	I want the cheapest guesthouses shown first so I can spot affordable options quickly.
Customer	I want to save favourites so I don't have to repeat my search later.
Customer	I want to rate and comment on a guesthouse so I can share my experience.
Customer (proposed)	I want to sign in with my Google account so I don't have to create and remember a separate password.
Admin	I want to log in securely so only authorised staff can manage listings.
Admin	I want to register a guesthouse with its images, amenities and location so customers can find it.
Admin	I want a dashboard with key stats so I can see how the platform is doing at a glance.

8. Main Use Cases

Use Case	Actor	Precondition	Main Flow
Search Guesthouses	Customer	None	Enter a name or location; system returns matches, cheapest first by default.
View Guesthouse Details	Customer	Listing exists	Select a listing; system shows gallery, description, amenities, price, contact and map.
Favourite a Guesthouse	Customer	Logged in	Tap favourite; system saves the customer–guesthouse link.
Rate / Comment	Customer	Logged in	Submit a rating and/or comment; system validates and stores it, and updates the average rating.
Register a Guesthouse	Admin	Logged in	Enter details and upload images; system validates and stores the listing.
Create Admin Account	Admin	Logged in	Enter new admin details; system validates and creates the account.

CUSTOMER

Search • View Details • Favourite • Rate • Comment

ADMIN
Login • Register Guesthouse • Manage Admins • View Dashboard

9. Data Model

We're using a NoSQL, document-based model (MongoDB), since guesthouse listings naturally vary different numbers of gallery images, different amenities, and so on. There are four main collections.

guesthouses

- name, logo, gallery (array of images), description, amenities
- pricePerNight, city, country, location {latitude, longitude}
- contact {phone, email}, createdBy (admin reference), createdAt

logo and gallery store file paths (e.g. /uploads/guesthouse123-1.jpg) pointing to images saved on the backend server, rather than links to a paid cloud storage service.

users

- name, email, passwordHash, role (admin or customer), createdAt
- authProvider ("local" or "google"), googleId — optional, only set if the customer signed up with Google (proposed enhancement, see Section 3)

ratings

- guesthouseId, customerId, rating (1–5), comment, createdAt

favourites

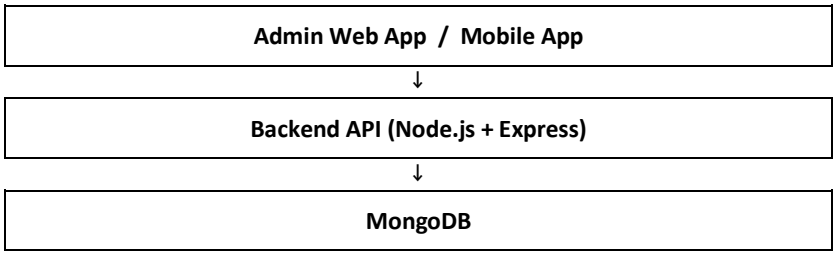
- customerId, guesthouseId, createdAt

How they connect

A guesthouse is created by one admin and can have many ratings and favourites. Each rating or favourite links one customer to one guesthouse. Instead of joins like a relational database, the backend just queries ratings/favourites by guesthouseId or customerId, and averages the ratings for the guesthouse's display page.

10. System Architecture

Simple client–server setup. Both apps talk to one backend API, and only the backend touches the database, so the two clients stay simple and the data stays consistent.



- Google Maps Platform — for capturing and showing guesthouse coordinates.
- Image storage — logos and gallery images are saved directly on the backend server's disk (a local /uploads folder), and the database only stores the file path. This avoids any paid cloud storage service, which isn't necessary for a project of this size.

11. Proposed Technology Stack

These are proposed technologies — not all of them are implemented yet.

Component	Technology	Why
Admin Web	React + TypeScript	Required by the brief; TypeScript helps catch mistakes early across a group codebase.
Mobile App	React Native + Expo	Reuses our React/TypeScript knowledge, and Expo makes testing on real phones easier.
Backend	Node.js + Express + TypeScript	Lightweight, well documented, and consistent with the rest of the stack.
Database	MongoDB	NoSQL as required, and its flexible documents suit varying guesthouse data well.
Maps	Google Maps Platform	Well-supported way to capture and display coordinates.
Image Storage	Local server storage (Express static /uploads folder)	Free — avoids a paid cloud storage subscription, which isn't needed at this scale. The database stores only the file path.
Authentication (proposed)	Google OAuth via Firebase Authentication (free tier)	Optional sign-in method proposed alongside standard email/password login — see Section 3 and the note under Functional Requirements.
Version Control	GitHub	Required by the brief; used to coordinate our work.

12. Security, Testing and Methodology

Security

- Passwords hashed (e.g. bcrypt) — never stored as plain text.
- Admin only and customer only actions require a valid login token.
- Backend validates input from both apps before saving anything.
- API keys and database credentials kept out of source control, using a .env file.
- If Google sign-in is added (proposed), Google's OAuth token is verified server side before a session is created we never handle the customer's Google password directly.

Testing

- Unit tests for logic like sorting, average ratings, and auth.
- API testing with a tool like Postman for key endpoints.
- Manual testing of both apps against the functional requirements.
- Checking that admin entered data shows up correctly on the mobile app.

Methodology

We're building this iteratively auth and guesthouse registration first, since everything else depends on them, then search/sorting, then favourites/ratings/comments and the dashboard. Work is split into GitHub feature branches merged into a shared development branch once reviewed.

13. Project Team

We split the team by strength rather than assigning everyone a bit of everything: three of us are focused on backend, two on frontend (one on each app), and two on the API layer connecting them. This came out of a

quick poll within the group, and it also satisfies the brief's requirement that work across the admin and mobile sides be clearly divided.

Team Member	Student Number	Main Responsibility
Tsholofelo Blessing Bathusi	24019716	Backend development — server-side logic and database integration
Helen Tlhomamo Mahule	24019725	Backend development — database design and data access logic
Pako Tumelo Ikobeng	24019736	Backend development — authentication and guesthouse management logic
Peo Abigail Motswasele	24020099	Frontend development — mobile app screens and search/browse UI
Angel Mhaladi	24019850	Frontend development — admin web portal UI and dashboard
Pako Serumola	24019735	API development — building and documenting the endpoints both apps rely on
Lefa Kato Ramonameng	24020066	API development — endpoint integration and testing

How the Groups Work Together

- Backend team (Tsholofelo, Helen, Pako I.) — owns the database schema, business logic, and the core server that both apps depend on.
- Frontend team (Peo, Angel) — Peo builds the mobile app screens for customers; Angel builds the admin web portal and its dashboard.
- API team (Pako S., Lefa) — defines and documents the endpoints connecting the frontend apps to the backend, and tests that data flows correctly between all three layers.

14. Requirements Traceability

A quick map of brief requirements to where they're covered in this document.

Assignment Requirement	System Feature	Document Section
Admin login & extra admin accounts	Admin authentication	Functional Requirements
Guesthouse registration (name, logo, gallery, amenities, coordinates, price, city, country, contact)	Guesthouse management	Functional Requirements, Data Model
Admin dashboard stats	Dashboard	Functional Requirements
Browse without account / search by name & location	Public browsing & search	Functional Requirements
Sort by price/rating, cheapest first	Search sorting	Functional Requirements
Customer login, favourites, ratings, comments	Customer account features	Functional Requirements
Shared data between apps	Central backend & database	System Architecture
No booking / payment / reservation	Explicitly excluded	Scope and Limitations

15. Deliverables and Conclusion

- Source code for the admin web app and the mobile app.
- MongoDB collection structure (Section 9).
- A README for each part covering setup and environment variables (placeholders, not real secrets).
- A data model diagram and a contribution log.
- A live demo or demo video covering both apps.

This project is our answer to a fairly ordinary problem: guesthouse information is scattered, and it's hard to compare options before you've even arrived somewhere. Splitting the system into an admin portal and a mobile app keeps each side simple while still covering everything the brief asks for. We've deliberately left out booking, payment and reservations, since this is a discovery platform, not a booking one and we've tried to keep that boundary clear throughout this document rather than just stating it once and moving on.